

Echolin.ai: An Accessible Web Platform for Image and Video Deepfake Detection Using a Pretrained Vision Transformer

Kaustubha Venkata Eluri
Northeastern University
eluri.k@northeastern.edu

Abstract—The proliferation of AI-generated synthetic media (“deepfakes”) has outpaced the availability of easy-to-use, non-specialist verification tools. This paper documents Echolin.ai (in-product brand: DeepShield AI), a full-stack web application that classifies user-uploaded images and videos as “Real” or “Fake” using a pretrained Vision Transformer (ViT), with an optional large-language-model (LLM) layer that translates the raw verdict into a plain-English explanation. The system is built on a React/TypeScript frontend, a Flask backend serving the live inference path, and Supabase for authentication, persistence, and row-level-secured storage. Video is handled through frame sampling and majority voting across the first ten frames of a clip. Rather than claiming novel detection accuracy, this paper focuses on the system-engineering contribution: composing an off-the-shelf, pretrained classifier into a coherent, transparently-documented product, including the explicit architectural decisions and known limitations recorded by the project’s own documentation. We report the project’s honestly-disclosed evaluation posture — reliance on the upstream model card’s published accuracy figures without independent re-benchmarking — and discuss the engineering trade-offs (pretrained-model reuse, frame-sampling strategy, synthetic “artifacts” field, framework choice, monolithic frontend component) that shaped the system.

Index Terms—deepfake detection, vision transformer, synthetic media, human-computer interaction, software architecture, interpretability

I. INTRODUCTION

Generative models capable of producing photorealistic synthetic images, video, and audio have become widely accessible, and reported deepfake-related fraud has grown sharply in recent years; identity-verification vendors Sumsu and Onfido have reported an approximate 2,100% rise in deepfake fraud incidents between 2019 and 2023. This trend has outpaced the availability of simple, publicly accessible verification tools: existing deepfake-detection capabilities are frequently either locked behind enterprise contracts or distributed as research code requiring substantial setup effort, leaving journalists, content moderators, educators, and everyday users without a fast way to ask a basic question — “is this real?”

Echolin.ai addresses this gap by providing a single-screen web tool: a user uploads an image or video and receives a Real/Fake verdict with an associated confidence score, together with an optional natural-language explanation generated by a large language model. The system is explicitly scoped as a *detection-only* platform; it does not generate synthetic media, and its authors describe it as one signal to be used alongside

provenance checks, reverse image search, and human review rather than as a final arbiter of authenticity.

This paper documents the system as it is actually built and evaluated, drawing exclusively on the project’s own README and internal documentation (architecture notes, architecture decision records, a model card, a limitations document, and a portfolio case study). Rather than presenting new benchmark results, the paper’s contribution is a transparent account of (1) how a pretrained Vision Transformer is composed into a usable, authenticated, persisted web product; (2) the specific engineering trade-offs made along the way, most of which are documented by the project team as deliberate, tracked technical debt rather than oversights; and (3) an honest statement of what the system does and does not achieve, including the fact that no independent accuracy benchmarking was performed.

II. RELATED WORK

Automated detection of manipulated or AI-generated visual media has largely been approached as a binary or multi-class image/video classification problem. Early deep-learning approaches relied on convolutional neural networks (CNNs) trained to pick up on generator-specific artifacts such as inconsistent blending boundaries, unnatural color statistics, or frequency-domain irregularities introduced by upsampling operations in generative adversarial networks. More recently, transformer-based vision architectures have been applied to the same task. The Vision Transformer (ViT) architecture, which treats an image as a sequence of patch embeddings processed by a standard transformer encoder, was introduced by Dosovitskiy et al. as a general-purpose image classification backbone and has since been adapted, via transfer learning and fine-tuning, to a range of downstream classification tasks, including deepfake detection, by treating the problem as binary classification over real and fake image patches or frames.

A recurring theme in the applied deepfake-detection literature and in practitioner-facing tools is the distinction between (a) building or improving a detection model itself, which requires large labeled corpora, substantial compute, and continual retraining as generator architectures evolve, and (b) building the surrounding system — ingestion, inference serving, explanation, authentication, and persistence — that turns a classifier into something usable by a non-technical end user. Video-based detection introduces the additional question

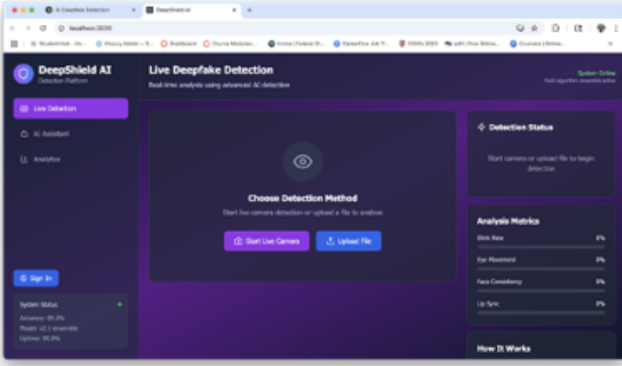


Fig. 1. The live detection interface, showing the drag-and-drop upload surface and the real-time Real/Fake verdict with associated confidence score returned by the `/api/detect` endpoint.

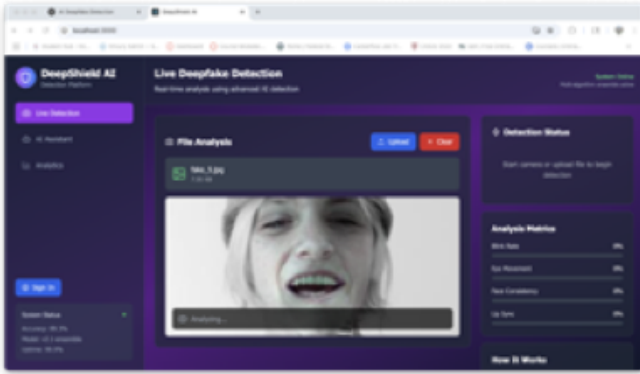


Fig. 2. The file analysis view, presenting per-upload detection metrics alongside the synthetic artifacts field discussed in Section III-C.

of temporal aggregation: whether to classify a single representative frame, every frame, or some sampled subset, and how to combine per-frame predictions into a single verdict. Echolin.ai sits squarely in category (b): it reuses a third-party pretrained ViT classifier without modification and focuses its engineering effort on frame-sampling strategy, failure containment between detection and explanation subsystems, and transparent user-facing communication of model confidence and limitations, rather than on advancing detection accuracy itself.

III. SYSTEM ARCHITECTURE

Echolin.ai is organized as a three-tier system: a React/TypeScript frontend, a Flask backend that performs model inference, and a Supabase-managed layer for authentication, relational storage, and file storage.

A. Frontend

The frontend is implemented in React 19 with TypeScript 4.9 and styled with Tailwind CSS. The user-facing surface includes drag-and-drop file upload (with batch support), a live detection status sidebar, an analysis metrics panel, a real-time chat interface with the AI assistant, and a settings modal

(Fig. 1). Discrete concerns are factored into components such as `AuthModal`, `ChatHistory`, `SettingsModal`, `UploadComponent`, and `UserProfile`, while three frontend services (`supabaseService.ts`, `openaiService.ts`, `LLMService.js`) encapsulate authentication state, database access, and LLM calls, respectively. The root `App.tsx` component, at 1,808 lines, retains a large share of application state and is documented internally as a known architectural issue (Section IV).

B. Backend and Inference

The live request path is served by a Flask API (`backend/app.py`) exposing two endpoints: `POST /api/detect`, which returns a label and confidence score only, and `POST /api/agent-detect`, which additionally invokes an LLM-based explanation step. Uploaded files are sent as multipart form data from the frontend.

Model inference is performed in `backend/detector.py` using the Hugging Face model `ashish-001/deepfake-detection-using-ViT`, a ViT-Base architecture (approximately 86 million parameters) fine-tuned upstream for binary classification over {Real, Fake}. The image processor and model are loaded once as a module-level singleton at server startup, so that per-request latency reflects only preprocessing and the forward pass. For a single image, the pipeline is: decode via PIL, preprocess with the Hugging Face `AutoImageProcessor`, run a `torch.no_grad()` forward pass through `AutoModelForImageClassification`, and apply softmax to obtain a label and a confidence score (the maximum class probability). Reported latency is approximately 0.5 seconds per image on CPU.

For video, `detect_video` uses OpenCV (`cv2.VideoCapture`) to extract up to ten frames sequentially from the start of the file (a constant `MAX_VIDEO_FRAMES = 10`), passes each frame through the same per-image ViT pipeline, and produces a final label by majority vote across the ten frame-level predictions. The reported confidence score is the average confidence over only the frames that voted with the winning label, not all ten frames. This entire pipeline runs in approximately five seconds on CPU.

A separate `detection_model/` directory hosts a parallel FastAPI service used for offline, batch-style experimentation (via `deepfake_image.py`, `deepfake_video.py`, `fast_api.py`, and two Jupyter notebooks documenting exploratory analysis). This service is explicitly documented as not being part of the live production request path; it exists to keep the model-evaluation workflow reproducible and separated from the deployed API.

C. Interpretability Framing

As surfaced in the file analysis view (Fig. 2), the JSON response returned by the detection endpoints includes an `artifacts` field, e.g. `[{"type": "face_texture", "score": confidence * 100}]`. The project's own

documentation is explicit that this field is *not* a genuine artifact-localization output: the underlying ViT is a single binary classifier that produces one softmax probability with no spatial or sub-feature attribution, and the `artifacts` entry is a relabeled repackaging of that same confidence value, included so the frontend has a uniform field to render. The documentation frames a future upgrade path toward genuine interpretability — either a Grad-CAM-style attention/saliency map over the ViT’s patch embeddings, or an ensemble of single-purpose detectors (e.g., blink-rate, lip-sync, or frequency-domain analyzers) — as future work rather than a currently implemented capability.

D. LLM Explanation Layer

When a request is made to `/api/agent-detect`, `agent.py` chains the detection result, the original filename, and a requested technical level into `llm_service.generate_analysis_explanation()`, which calls an OpenAI-compatible chat-completions endpoint. Four system-prompt modes are supported: `analysis` (forensic-style), `educational` (concept-first), `conversational` (free-form Q&A), and `threat_analysis` (landscape-level framing). The endpoint is configurable via environment variables (`GMI_API_URL`, `GMI_API_KEY`), defaulting to GMI Cloud (a Northeastern-affiliated AI cloud offering free student credits) with OpenAI’s `gpt-3.5-turbo` as a drop-in fallback, since both expose an OpenAI-compatible REST surface. If the explanation call fails for any reason (missing key, network error, rate limiting), the exception is caught and the response instead includes the string “LLM explanation unavailable. Core detection succeeded.”; the binary classification result is unaffected.

E. Persistence and Identity

Authentication is handled by Supabase Auth using JWT tokens and email/password sign-in via `@supabase/auth-ui-react`. Application data lives in three Postgres tables: `chat_sessions`, `chat_messages`, and `detections`, each governed by row-level security (RLS) policies that restrict every row to `auth.uid() = user_id`. A Supabase Storage bucket named `uploads` holds submitted files; auto-deletion is opt-in rather than default. Because RLS enforcement happens at the Postgres layer rather than in application code, the frontend can safely use the Supabase anonymous key for queries.

IV. DESIGN DECISIONS

The project maintains a set of dated architecture decision records (ADRs) that make its engineering trade-offs explicit. The most consequential are summarized below.

Pretrained model over custom training (ADR-001). The team chose to integrate an existing Hugging Face ViT model rather than train or fine-tune a custom detector. The stated rationale is that building a competitive deepfake detector requires a large non-overlapping labeled corpus, a GPU training budget,

and a continuous retraining loop to track evolving generator architectures — effectively a multi-quarter research effort in its own right. The pretrained model is accessed through a single function boundary in `detector.py`, so that swapping in a fine-tuned or ensemble model later is a localized, one-file change. The documented cost is that detection quality is capped by the training distribution of the third-party model, and generators released after that model’s training cutoff may not be reliably detected.

Two backend frameworks for two roles (ADR-002). Flask serves the two-endpoint live API, while a separate FastAPI service in `detection_model/` supports asynchronous batch processing and streaming responses for offline model evaluation. The team frames this as choosing the right tool per role rather than standardizing prematurely, while acknowledging the cost of maintaining two web frameworks in one repository; directory boundaries (only `backend/` is reachable from the frontend) mitigate the resulting confusion.

Ten-frame majority vote for video (ADR-003). Classifying a single frame was judged too noisy (a transient expression or lighting artifact could flip the verdict), while classifying every frame of a full video was judged too expensive (a 30-second, 30fps clip would require roughly 900 forward passes, on the order of 7.5 minutes on CPU). Sampling the first ten frames sequentially, majority-voting the label, and averaging confidence only over frames that agree with the winning label was chosen as a deterministic, reproducible middle ground that completes in about five seconds. The documented cost is that deepfakes confined to frames beyond the tenth sampled frame may be missed; the team’s justification is that real-world deepfakes tend to be frame-consistent because the underlying generator is typically applied uniformly across a clip.

Honestly-labeled synthetic artifacts field (ADR-004). As discussed in Section III-C, the `artifacts` field was retained from an earlier UI design that anticipated per-artifact breakdowns before the team had selected a model. Rather than remove the field or silently misrepresent it, the team chose to document it explicitly as synthetic in both the architecture notes and the limitations document, treating this as an acceptable technical-debt decision precisely because it is disclosed rather than dressed up as genuine localization.

Non-blocking LLM explanation (ADR-005). The explanation layer is treated as a best-effort enhancement layered on top of the load-bearing binary classification. Any failure in the LLM call is caught and converted into a graceful fallback message rather than propagated as a request failure, so that the reliability of the core detection result is decoupled from the reliability of a third-party LLM API.

Deferred frontend decomposition (ADR-006). The 1,808-line `App.tsx` component is explicitly acknowledged as technical debt. The team’s stated plan is to extract `useDetection()`, `useChat()`, and `useAuth()` hooks and reduce `App.tsx` to a routing shell, but this refactor has been deferred because some of the component’s internal state interactions are subtle enough that a rewrite carries real regression risk against the working demo.

Managed backend-as-a-service over custom infrastructure (ADR-007). Authentication, relational storage, and file storage are all delegated to Supabase rather than built from scratch (custom Postgres, custom auth, custom S3-compatible storage). The stated reasoning is that the project’s value proposition is the detection capability, not novel identity or storage infrastructure, and that building the latter from scratch would be “displacement activity.” The documented cost is vendor lock-in, judged acceptable because the entire integration surface is confined to a small number of files under `src/services/`.

Pluggable LLM endpoint (ADR-008). The explanation layer is configured entirely through environment variables so that GMI Cloud (offering free credits to Northeastern students) can serve as the default provider with OpenAI as a drop-in fallback, since both expose OpenAI-compatible chat-completions REST APIs. This avoids forking the codebase per provider.

Naming debt between repository and product (ADR-009). The GitHub repository retains the name `Echolin.ai` from an earlier phase of the project, while the shipped product identity is “DeepShield AI” (team name ICE-CUBA). The team explicitly records this divergence as intentional short-term naming debt, preserving the original repository URL to avoid link rot in already-shared course materials, with reconciliation deferred to a future domain or app-store launch.

V. EVALUATION

Echolin.ai’s own documentation is explicit that no independent, project-run accuracy benchmarking of the detection pipeline has been performed. The model card and limitations document both state that the only published accuracy figures for the underlying classifier — reported as 92–98% on standard test sets — originate from the upstream Hugging Face model card for `ashish-001/deepfake-detection-using-ViT`, and that these figures have not been independently re-evaluated by the project team on held-out or out-of-distribution data. The documentation explicitly instructs that these published numbers should be read as “upper bounds on optimistic test conditions,” not as production guarantees, and notes that real-world distribution shift — compression artifacts, social-media re-encoding, and screen-recorded fakes — would be expected to degrade them further.

Rather than a formal quantitative evaluation, the project’s evaluation posture as documented is qualitative and design-oriented: the team reasons explicitly about failure modes (LLM unavailability, malformed uploads, videos with deepfake content confined to unsampled frames) and designs containment behavior for each — for example, returning a fallback explanation string rather than failing the request outright when the LLM call errors, and defaulting to CPU inference with a documented 0.5-second per-image and 5-second per-video latency. The system also documents a recommended *verification workflow* rather than presenting the classifier as a sufficient standalone signal: a provenance check (e.g., C2PA/Content

Credentials, where present), reverse image search, source-trail verification, the Echolin.ai detection step, and human review when any of the preceding steps conflict. This is presented as guidance for interpreting a high-confidence “Fake” verdict as sufficient grounds to escalate for further review, but explicitly not as sufficient grounds on its own to publish a correction or take a moderation action.

Consistent with the strict evidentiary basis of this paper, we do not report any accuracy, precision, recall, or F1 figures for Echolin.ai’s detection pipeline beyond the upstream, non-project-verified 92–98% range disclosed in the model card, and we treat the absence of independent benchmarking as a limitation of the evaluation rather than a gap to be papered over.

VI. LIMITATIONS

The project’s own `limitations.md` enumerates a substantial set of known constraints, which we reproduce and organize below rather than restate in weakened form.

Model-level limitations: the system relies on a single pre-trained ViT with no ensembling or model-vs-model corroboration; its accuracy is bounded by the coverage of its (unknown, third-party) training set, so generator architectures released after that model’s training cutoff may evade detection without any explicit signal that this has occurred; it provides no per-region localization, i.e., it cannot indicate which part of an image or frame was manipulated; and, as discussed above, its reported accuracy figures are hedged because they are inherited from the upstream model card rather than independently verified.

Video-level limitations: only the first ten frames of a video are analyzed, so manipulations confined to later frames can be missed entirely; no temporal-coherence modeling is performed, meaning the pipeline does not exploit frame-to-frame consistency, which the team itself identifies as one of the strongest available deepfake signals; and there is no audio-analysis path, so audio-only or lip-sync-based manipulations are entirely out of scope.

Pipeline-level limitations: inference runs on CPU only, with no GPU acceleration on the live path; there is no batching, so throughput is not optimized for concurrent load; and there is no streaming inference, meaning the full file must be uploaded before any detection can begin, precluding real-time analysis of a live video stream.

System-level limitations: the LLM explanation feature is optional and best-effort by design, so explanations can silently be omitted under provider failure; multi-tenant isolation is limited to per-user row-level security, with no team- or organization-level workspace concept; and uploaded files are not deleted automatically by default, requiring an operator-configured retention policy.

Trust and operational limitations: the team frames detection as one signal among several rather than a definitive verdict, explicitly warning that both false positives and false negatives occur; error handling is described as “demo-grade,” with some upload edge cases (e.g., corrupted EXIF metadata, unusual

codecs) surfacing as unhandled server errors rather than user-friendly messages; there is no rate limiting on the Flask backend, which the team notes should sit behind a gateway in production; and logging currently relies on ad hoc `print()` statements rather than structured, production-grade logging.

Finally, the `synthetic artifacts` field (Section III-C, ADR-004) is itself a limitation on the system’s interpretability claims: end users are shown a field that resembles a fine-grained artifact breakdown, and while the project documents this honestly in its own materials, a user interacting with the product UI alone, without reading the documentation, could reasonably over-interpret this field as genuine localized evidence.

VII. CONCLUSION AND FUTURE WORK

Echolin.ai demonstrates that a usable, authenticated, end-to-end deepfake-screening web application can be assembled by composing a pretrained Vision Transformer with conventional web-application infrastructure (React, Flask, Supabase) and an optional LLM explanation layer, without requiring the team to train or fine-tune its own detection model. The project’s central contribution, as documented, is not a novel detection method but a disciplined systems-engineering exercise: the architectural seam around the model is kept narrow and swappable, failure modes between subsystems (detection vs. explanation) are explicitly contained, and — notably — every known shortcut and piece of technical debt (a synthetic artifacts field, a monolithic frontend component, unverified upstream accuracy claims, a ten-frame video-sampling cap) is documented candidly rather than concealed.

The project’s own roadmap lays out a phased path toward closing the gaps identified in Section V: Phase 1 targets a detection upgrade, including replacing fixed first-ten-frame sampling with stratified sampling across a video’s full duration, adding a temporal-coherence head (e.g., a lightweight LSTM or attention mechanism over frame-level features) to exploit frame-to-frame consistency, introducing an audio path for lip-sync and voice-clone detection, and replacing the synthetic artifacts field with genuine Grad-CAM-derived attention maps or a dedicated localization model. Phase 2 targets decomposing the monolithic `App.tsx` into dedicated hooks and adding component-level tests. Later phases address operational hardening (structured logging, rate limiting, error monitoring, containerized local development), a browser extension for in-context verification, an open API for researchers and journalists, and a detection-trend dashboard. The team’s roadmap explicitly excludes deepfake generation and forensic-grade chain-of-custody evidence handling as out of scope, preserving the platform’s stated identity as a detection-only, single-signal verification tool.

ACKNOWLEDGMENT

The author thanks Northeastern University and the GMI Cloud program for providing computing credits used during development.

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [2] K. V. Eluri, “Echolin.ai (DeepShield AI),” GitHub repository, README and internal documentation (`docs/architecture.md`, `docs/decisions.md`, `docs/limitations.md`, `docs/model-card.md`, `docs/roadmap.md`, `docs/portfolio-case-study.md`), 2026.
- [3] ashish-001, “deepfake-detection-using-ViT,” Hugging Face Model Hub. [Online]. Available: <https://huggingface.co/ashish-001/deepfake-detection-using-ViT>